SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration:1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure

Scenario Based Assignment

Code of Conduct:

I P Navya (1925260224) (*Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **P Navya**

1. Emergency Medicine Delivery Drones

Scenario: AI-powered drones deliver emergency medicines in a flood-affected region. Roads can become blocked, weather affects travel cost, and the drone receives only partial real-time information.

i. Greedy Best-First Search: This algorithm selects the node with the smallest heuristic value h(n), which appears closest to the goal. It can make decisions quickly, so it may be useful when medicine must be delivered urgently. However, it ignores the cost already spent and may choose a route with difficult terrain or a blocked road. Therefore, the route may not be the safest or shortest.

ii. A* Search: A* uses f(n) = g(n) + h(n). Here, g(n) is the actual cost from the starting point and h(n) is the estimated cost to the goal. It considers both the distance already travelled and the remaining estimated distance. With an admissible heuristic and a stable search graph, A* can find an optimal route, but it may take more time and memory than Greedy Best-First Search.

iii. Impact of heuristic accuracy: A more accurate heuristic guides the search toward a good route. Greedy Best-First Search depends strongly on the heuristic, so a poor estimate can mislead it. A* also uses the heuristic, but it balances it with the actual path cost. If the heuristic overestimates the remaining cost, A* may lose its optimality guarantee.

iv. Effect of blocked paths: If a road becomes blocked, the current route may no longer work. The drone must update its map and calculate another route. Frequent changes increase computation, so the system should support quick updates and re-planning.

v. Recommendation: I would recommend A* with regular map updates and re-planning because it considers both path cost and estimated distance. For real-time use, the drone should also apply safety checks and switch to an alternative route quickly when a road or area becomes unsafe.

2. Smart-City Traffic Signal Optimization

Scenario: A smart city adjusts traffic signals to reduce waiting time, fuel consumption, and congestion. Traffic changes frequently, and the number of possible signal plans is very large.

Formulation as an optimization problem: Each possible set of signal timings is a candidate solution. The objective is to minimize total waiting time, fuel use, and congestion while following traffic-safety rules. A neighbouring solution can be created by changing the duration or order of a signal phase. Exhaustive search is inefficient because the number of combinations grows rapidly and traffic conditions keep changing.

i. Hill Climbing: Hill Climbing begins with an initial signal plan and repeatedly selects a neighbouring plan that improves the objective. It is simple and fast, but it can stop at a local optimum even when a better overall plan exists.

ii. Local maxima, plateaus, and ridges: A local maximum is a plan that is better than nearby plans but is not the best possible plan. A plateau occurs when many neighbouring plans have similar results, so the algorithm cannot identify a clear direction. A ridge occurs when improvement requires several coordinated changes, while changing only one signal does not appear helpful.

iii. Simulated Annealing or Genetic Algorithms: Simulated Annealing sometimes accepts a worse solution during the early search, which helps it escape local optima. This becomes less likely as the search continues. A Genetic Algorithm keeps several candidate plans and uses selection, crossover, and mutation to create new plans. Both methods explore more possibilities than basic Hill Climbing.

iv. Recommendation: I would choose a Genetic Algorithm for a large traffic network because it explores many candidate plans. The system should update traffic data regularly, use a limited time for each optimization cycle, and ensure that every solution follows safety constraints. The results should be checked against the current signal plan before being applied.

3. Autonomous Mars Rover

Scenario: A rover explores unknown Martian terrain, collects samples, avoids hazards, and makes decisions with limited sensing and no complete map.

i. Why an online search agent is needed: An online search agent makes decisions while interacting with the environment. The rover observes its surroundings, moves, receives new information, and updates its plan. Offline search is less suitable because the rover does not know the complete map before it starts exploring.

ii. Partial observability and dynamic environments: The rover's sensors may not detect every rock or crater, so its information can be incomplete. Terrain estimates may also be uncertain. The rover must account for these risks, limited energy, and possible communication delays by moving carefully and planning again when new information is received.

iii. Building and updating the internal model: The rover stores information about explored locations, detected obstacles, possible routes, and estimated travel costs. After each movement, it uses sensor readings to update its map, marks blocked areas, and revises its route if needed.

iv. Comparison with uninformed and informed search: Uninformed search does not use a heuristic and may explore many unnecessary locations. Informed search uses a heuristic to guide the search toward the goal. Online search is different because the agent learns about the environment while acting. An online agent can also use heuristic guidance as new information becomes available.

v. Recommendation: I would use an online, heuristic-guided search approach with hazard detection and frequent re-planning. It is suitable because the rover can adapt to new observations, avoid unsafe paths, and make decisions without a complete map.

4. University Exam Timetabling as a Constraint Satisfaction Problem (CSP)

Scenario: A university must schedule exams while avoiding student clashes and respecting room limits, subject order, faculty availability, and special accommodations.

Formulation as a CSP: A Constraint Satisfaction Problem contains variables, domains, and constraints. The variables can be the time slot and room for each exam. The domains are the available time slots and suitable rooms. The constraints include no overlapping exams for the same student, room capacity, room availability, subject-order requirements, faculty availability, and accommodation needs.

i. Variables, domains, and constraints: Each course is represented by a variable. Its domain contains the feasible time-slot and room combinations. Constraints remove combinations that cause student clashes, exceed room capacity, conflict with faculty availability, or violate the required subject order.

ii. Backtracking search: Backtracking assigns a time and room to one exam and checks whether the assignment violates any constraint. If it is valid, the algorithm moves to the next exam. If no available assignment works, it returns to an earlier exam and tries another choice.

iii. Constraint propagation: Forward checking removes values from the remaining variables' domains when they become invalid because of a new assignment. For example, once an exam is assigned to a time slot, that slot can be removed from the domain of exams that conflict with it. This detects problems early and reduces unnecessary searching.

iv. Real-world challenges and scalability: The number of courses, students, rooms, and constraints can make the problem large. I would use backtracking with forward checking, choose the most constrained exam first, and try the least restrictive valid assignment first. For very large timetables, the problem can be divided into smaller parts and the final timetable should be checked against all hard constraints.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

Scenario: A gaming company is developing an AI opponent that must choose good moves against human players while working under strict time limits and searching a large game tree.

i. How Minimax models the problem: Minimax represents possible moves as a game tree. At a maximizing level, the AI selects the move with the highest utility. At a minimizing level, it assumes the opponent will choose a move that reduces the AI's utility. The values are calculated from terminal states or estimated using an evaluation function when the search reaches its depth limit.

ii. Computational challenges: If each state has b possible moves and the search depth is d, Minimax may examine about O(b^d) nodes. This grows quickly and can exceed the available time. A large tree also requires a good evaluation function to estimate states that are not terminal.

iii. How Alpha-Beta pruning improves efficiency: Alpha-Beta pruning keeps track of alpha, the best value found for the maximizing player, and beta, the best value found for the minimizing player. If a branch cannot change the final decision, the algorithm stops exploring that branch. This saves computation without changing the Minimax result. With good move ordering, its best-case time complexity is approximately O(b^(d/2)); the worst case is O(b^d).

iv. Evaluation function: An evaluation function could combine resource levels, unit strength, territory controlled, progress toward objectives, defence, and mobility. Positive weights can reward strong positions, while penalties can represent lost units or exposed bases. The weights should reflect the game's goals, and the function should be quick to calculate.

v. Recommendation: I would use depth-limited Minimax with Alpha-Beta pruning, good move ordering, and a fast evaluation function. Iterative deepening allows the AI to keep the best move found if the time limit is reached. A transposition table can also avoid evaluating the same state repeatedly. These methods balance decision quality with real-time performance.